

Imputación de Datos Faltantes en Series Temporales Multivariantes IoT Con Modelos Transformer

Autor: Ricardo Ruiz Fernández de Alba

Tutores: María del Campo Bermúdez Edo

Daniel Bolaños Martínez



Granada, 8 de junio de 2025

Índice

1. Introducción y Problema
2. Fundamentos Teóricos
3. Arquitectura Transformer
4. Imputación: Métodos y SAITS
5. Caso de Estudio: Smart Poqueira
6. Resultados y Discusión
7. Conclusiones y Futuro

1. Introducción y Contexto

- **IA & IoT:** Transformación digital.
- **ML/DL:** Herramientas clave para datos.
- **Impacto:** Hacia la sostenibilidad (ODS).

1.2. El Desafío de los Datos Faltantes

- Contexto: Proyecto **Smart Poqueira (IoT)**.
- Datos: Series temporales de tráfico/contaminación.
- Problema: **Datos Faltantes** (`missing data`).
- Solución: **Imputación** con Transformers (SAITS).

1.3. Objetivos del Trabajo

Principal: Imputar datos IoT con modelos Transformer.

Específicos:

1. Fundamentación Teórica (Mat/ML/DL).
2. Análisis del Estado del Arte.
3. Desarrollo de Pipeline y Paquete Python.
4. Evaluación de Rendimiento.

2. Fundamentos Teóricos

- Bases: Álgebra Lineal, Análisis Matemático.
- Probabilidad y Estadística.
- Aprendizaje Automático y Profundo (ML/DL).
- Foco: Conceptos para modelos de atención (ej. Softmax).

2.5. La Función Softmax

Transforma un vector de valores reales en una distribución de probabilidad.

Softmax Can Be Tricky to Compute!

The Formula TheAiEdge.io

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Can overflow!

Exponentials are hard to compute!

$$e^{89} \approx 4.4 \times 10^{38} > \text{32-bit float limit: } 3.4 \times 10^{38}$$

Exponential can float-overflow!

We can use a trick: we compute the maximum value

$$m = \max \{x_1, x_2, \dots, x_N\}$$

And modify the original formula

$$\text{Softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}$$

Never overflow!

$$\frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}} = \frac{e^{-m} e^{x_i}}{e^{-m} \sum_{j=1}^N e^{x_j}} = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

This is mathematically equivalent but numerically stable!

(Infografía: TheAiEdge.io)

2.5. La Función Softmax

Transforma un vector de valores reales en una distribución de probabilidad.

Softmax Can Be Tricky to Compute!

The Formula TheAiEdge.io

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Can overflow!

Exponentials are hard to compute!

$$e^{89} \approx 4.4 \times 10^{38} > \text{32-bit float limit: } 3.4 \times 10^{38}$$

Exponential can float-overflow!

We can use a trick: we compute the maximum value

$$m = \max \{x_1, x_2, \dots, x_N\}$$

And modify the original formula

$$\text{Softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}$$

Never overflow!

$$\frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}} = \frac{e^{-m} e^{x_i}}{e^{-m} \sum_{j=1}^N e^{x_j}} = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

This is mathematically equivalent but numerically stable!

La fórmula estándar:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

(Infografía: TheAiEdge.io)

2.5.1. Estabilidad Numérica de Softmax

- Problema: `overflow` con e^{z_i} para valores grandes.
- Solución: Restar el valor máximo $m = \max(\{x_1, \dots, x_N\})$ de los logit.

2.5.1. Estabilidad Numérica de Softmax

- Problema: `overflow` con e^{z_i} para valores grandes.
- Solución: Restar el valor máximo $m = \max(\{x_1, \dots, x_N\})$ de los logit.

Fórmula estable:

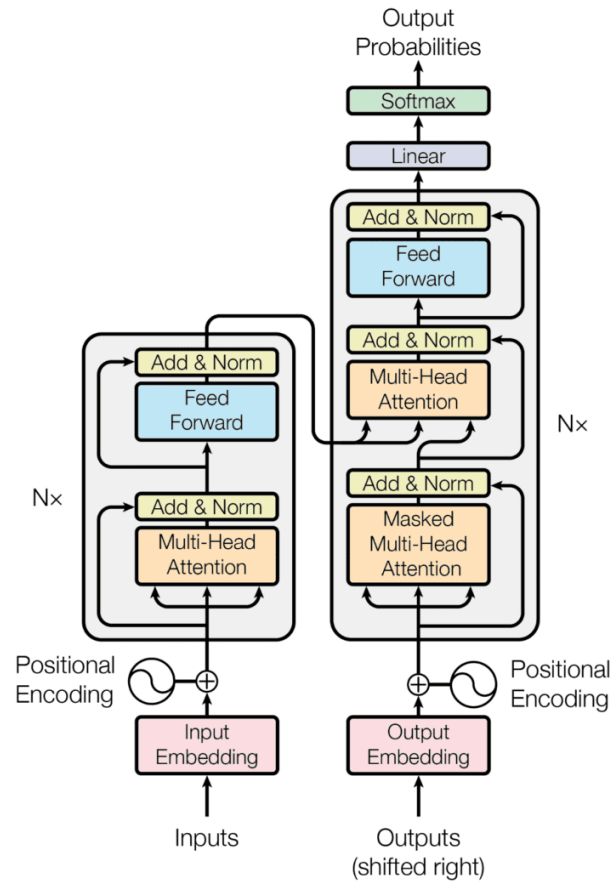
$$p_i = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}$$

- Resultado: Equivalente, pero numéricamente estable.

3.4. Evolución: Perceptrones → Transformers

- **Perceptrón:** Binario, lineal.
- **MLP/FFNN:** Capas múltiples, no linealidad.
- **RNN/LSTM:** Límites en secuencias largas.
- **Transformer:** Solución disruptiva.

3.4.3. Redes Profundas con Atención: Transformer



- Basado en **Atención**.
- Procesamiento **paralelo** de secuencias.

Componentes Transformer: Positional Encodings (PE)

- Provee información de orden y posición.
- Se suma a los embeddings de entrada.
- Generado por funciones $\sin$ y $\cos$.

Componentes Transformer: Self-Attention

- Idea clave: Scaled Dot-Product Attention.
- Genera **Q** (Query), **K** (Key), **V** (Value).
- Resultado: Representación contextualizada.

Componentes Transformer: Self-Attention

- Idea clave: Scaled Dot-Product Attention.
- Genera **Q** (Query), **K** (Key), **V** (Value).

Fórmula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Resultado: Representación contextualizada.

Componentes Transformer: Multi-Head Attention (MHA)

- Múltiples mecanismos de atención en paralelo.
- Cada "cabeza" aprende diferentes aspectos.
- Mayor capacidad para dependencias complejas.

Componentes Transformer: FFNN, Residual, LayerNorm

- **FFNN:** Añade no linealidad.
- **Conexiones Residuales:** Facilitan flujo de gradientes.
- **Normalización de Capas:** Estabiliza entrenamiento.

Arquitectura Encoder-Decoder

- **Encoder:** Procesa secuencia de entrada.
- **Decoder:** Genera secuencia de salida.
- Clave: Atención Enmascarada, Atención Encoder-Decoder.

4. Imputación de Datos: Definición

El Problema

- Objetivo: Estimar faltantes $X_{missing} \rightarrow \hat{X}$.
- Evaluación: Comparación con X_{orig} **solo en posiciones artificiales** ($M_{artificial}$).

4.2. Métodos de Imputación

1. Información Local: (Ffill, Bfill, Interpolación Lineal)
2. Estacionarios: (Media, Mediana)
3. Predictivos (ML/DL): (Transformers, SAITS)

5.6. Métodos Simples Implementados

- Estadísticos: Media, Mediana (por muestra).
- Locales: Ffill, Bfill, Interpolación Lineal.
- Limitaciones: Ignoran estructura temporal.

5.6.3. Imputación: Transformer y SAITS

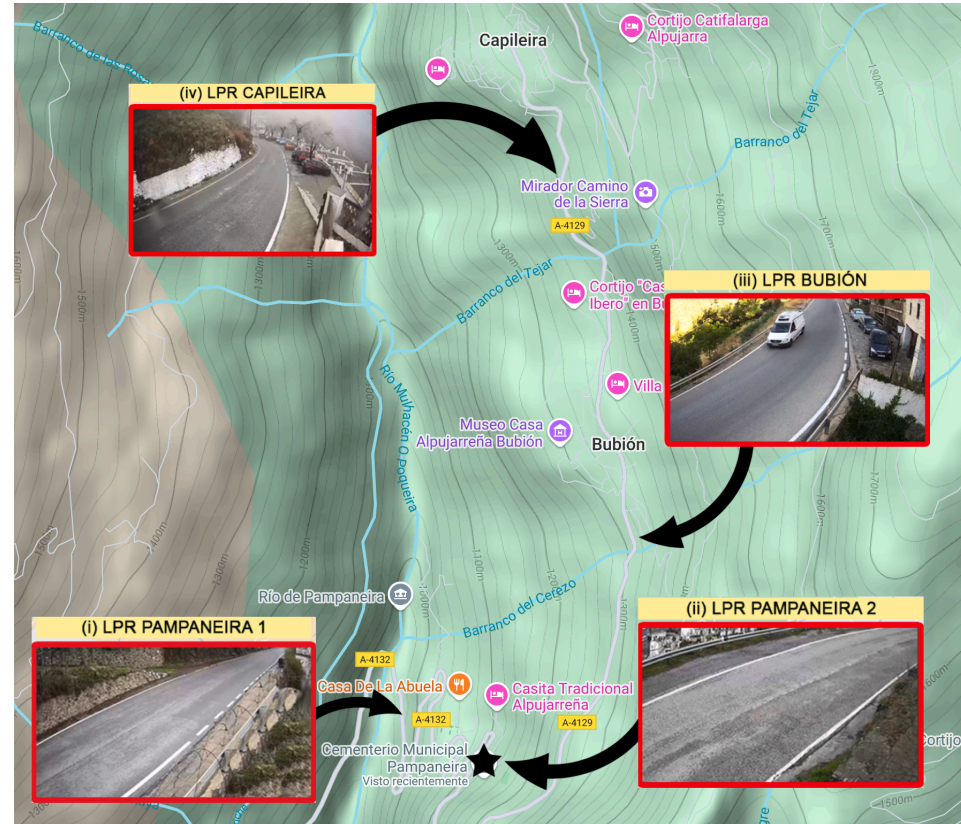
- Modelos DL: **Transformer** y **SAITS**.
- Librería: PyPOTS.
- Fases: **Entrenamiento** y **Imputación (Inferencia)**.

3.4.4. SAITS: Self-Attention for Time Series

- DL especializado en imputación de series temporales.
- Adaptaciones clave (Transformer):
 - DMSA (evita auto-atención).
 - Optimización Conjunta (MIT, ORT).

5. Caso de Estudio: Smart Poqueira

- Proyecto de investigación UGR.



- Infraestructura IoT: Cámaras LPR (tráfico), unidad móvil (contaminación).

5.3. Análisis Exploratorio de Datos (EDA)

- Alta incidencia de faltantes (~70% en eBC_tot).
- Distribuciones asimétricas (contaminantes).
- Correlaciones significativas (ej. eBC_tot y NO2).
- Clave para imputación predictiva.

5.4. Entorno Experimental y Flujo de Trabajo

- Paquete Python: `pampaneira\_imputation`.
- Flujo (Jupyter Notebook):
 1. Preprocesamiento (escalado, ventaneo, **faltantes artificiales**).
 2. Aplicación de métodos.
 3. Evaluación cuantitativa y visualización.

4.3. Métricas de Evaluación

- **MAE** (Mean Absolute Error):

$$\text{MAE}(\hat{X}, X_{\text{orig}}, M_{\text{ind}}) = \frac{\|\hat{X} - X_{\text{orig}}\| \odot M_{\text{ind}}\|_1}{\|M_{\text{ind}}\|_1}$$

- **RMSE** (Root Mean Squared Error):

$$\text{RMSE}(\hat{X}, X_{\text{orig}}, M_{\text{ind}}) = \sqrt{\frac{\|(\hat{X} - X_{\text{orig}})^{\odot 2} \odot M_{\text{ind}}\|_1}{\|M_{\text{ind}}\|_1}}$$

- **MRE** (Mean Relative Error):

$$\text{MRE}(\hat{X}, X_{\text{orig}}, M_{\text{ind}}) = \frac{\|\hat{X} - X_{\text{orig}}\| \odot M_{\text{ind}}\|_1}{\|X_{\text{orig}}\| \odot M_{\text{ind}}\|_1 + \epsilon}$$

Menor valor = Mejor rendimiento.

5.8. Evaluación del Rendimiento: Periodo 1

Cuadro 2: Resultados - Periodo 1

Métr...	Med...	Mean	Linear	Ffill	Bfill	Tran...	Saits
RMSE	1,1342	1,0521	0,87...	1,0604	1,0487	0,6471	0,6370
MSE	1,2865	1,1069	0,7737	1,1244	1,0997	0,4187	0,40...
MAE	0,6910	0,7346	0,4621	0,5618	0,5554	0,3378	0,3291
MRE	0,9311	0,98...	0,6226	0,7569	0,74...	0,4552	0,44...

Clave: SAITS y Transformer: Menores errores, rendimiento superior.

5.8. Evaluación del Rendimiento: Periodo 2 y Beijing

Cuadro 3: Resultados - Periodo 2

Métr...	Med...	Mean	Linear	Ffill	Bfill	Tran...	Saits
RMSE	1,1103	1,0609	0,9252	1,0754	1,1018	0,7361	0,7345
MSE	1,2329	1,1255	0,8559	1,1565	1,2140	0,5419	0,5395
MAE	0,7177	0,7637	0,4915	0,5836	0,58...	0,40...	0,3962
MRE	0,9161	0,9747	0,6273	0,7470	0,75...	0,5167	0,5057

Validación Dataset Air-Quality (Beijing): Superioridad confirmada.

5.8. Discusión de Resultados

- DL (SAITS/Transformer) claramente superior.
- SAITS lidera en precisión.
- Interpolación Lineal: Mejor método simple.
- Robustez frente a faltantes y patrones complejos.

7.2. Cumplimiento de Objetivos

Objetivo Principal: Cumplido.

Objetivos Específicos:

- Fundamentación Teórica 
- Estado del Arte 
- Modelo de Vanguardia (SAITS) aplicado 
- Paquete de Software desarrollado 
- Evaluación de Desempeño 

7.3. Limitaciones

- Alcance de datos.
- Hiperparámetros no optimizados a fondo.
- Patrones de faltantes artificiales simplificados.
- Impacto en tareas posteriores no evaluado.
- Comparativa limitada a DL y métodos simples.

7.4. Futuro y 7.5. Conclusión Final

Trabajo Futuro:

- Mejora de modelos Transformer.
- Integración con análisis posteriores.
- Desarrollo de software robusto.
- Aplicación a ODS.

Conclusión Final:

- SAITS (Transformers) superior en imputación IoT.
- Bases para calidad de datos y sostenibilidad.

¡Gracias por su atención!

¿Preguntas?